

Informe Técnico: Arquitectura y Funcionamiento del Sistema Origen X

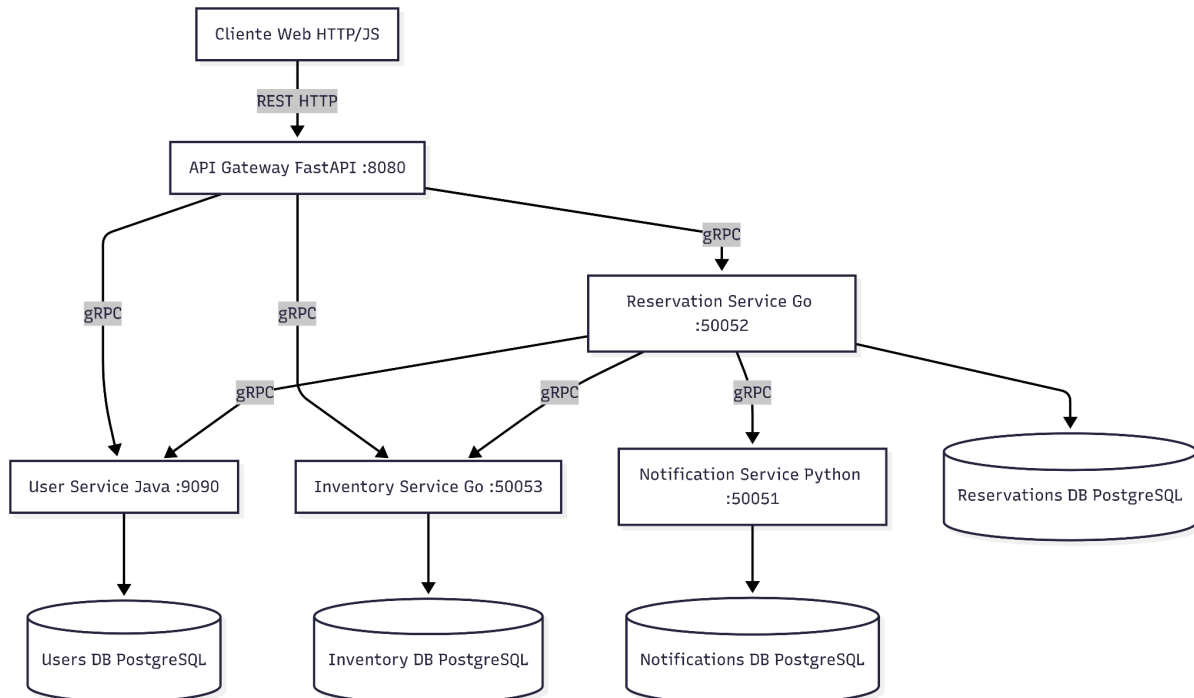
Benjamin Cruzado, Enzo Loren, Ignacio Morales, Paolo Paredes

1. Descripción del sistema

Origen X es una plataforma distribuida diseñada para la gestión del inventario hotelero y la confirmación de reservas en tiempo real. El sistema unifica la administración de catálogo y permite a los clientes consultar disponibilidad y confirmar alojamiento de manera concurrente. Está orientado a dos perfiles principales: clientes finales, que buscan y realizan reservas a través de una interfaz REST; y operadores del sistema, que requieren una infraestructura backend de alta disponibilidad, estricta consistencia en el stock y escalabilidad frente a la demanda concurrente.

2. Diagrama de arquitectura

El ecosistema opera bajo un entorno contenerizado (Docker) y utiliza una red privada aislada para la comunicación interna entre microservicios, exponiendo únicamente el API Gateway al exterior.



3. Decisiones técnicas y trade-offs

3.1 Patrón Database-per-Service (Aislamiento de persistencia)

- **Por qué se eligió:** Se decidió aprovisionar una instancia de PostgreSQL aislada para cada microservicio en lugar de utilizar una base de datos central compartida.
- **Qué se gana:** Se elimina el acoplamiento negativo por base de datos compartida (*Common Coupling*). Ningún servicio puede alterar las tablas de otro directamente, permitiendo escalar o migrar la capa de persistencia de cada dominio de manera totalmente autónoma.
- **Qué se sacrifica:** La gestión de infraestructura es más pesada (requiere múltiples volúmenes y contenedores). Se pierde la capacidad de utilizar uniones (JOINS) SQL nativas entre dominios y el cumplimiento ACID estricto en operaciones distribuidas.
- **Alternativas descartadas:** Base de Datos Monolítica compartida, descartada por representar un punto único de fallo y fomentar un fuerte acoplamiento estructural.

3.2 Inyección de datos (Data Coupling) hacia Notificaciones

- **Por qué se eligió:** El orquestador de reservas extrae el correo electrónico del usuario durante la validación de identidad y lo inyecta explícitamente en el payload gRPC hacia el servicio de notificaciones.
- **Qué se gana:** El servicio de notificaciones opera de forma 100% autónoma. Al no necesitar establecer conexiones de red de regreso hacia el servicio de usuarios para buscar correos, se erradica la dependencia cíclica y se reduce la latencia de red.
- **Qué se sacrifica:** El mensaje Protobuf central del servicio de reservas incrementa levemente su carga útil al tener que transportar datos que no son de su propio dominio transaccional (*Tramp Data*).
- **Alternativas descartadas:** Patrón de Contact Resolver en Notificaciones (Acoplamiento de Red), descartado porque condenaba el envío de correos a fallar si el servicio de usuarios sufría intermitencias.

3.3 Delegación asíncrona de sub-tareas (Fire-and-Forget en Go)

- **Por qué se eligió:** El orquestador de reservas despacha el comando de envío de correo encapsulando la llamada gRPC dentro de una *goroutine* no bloqueante, delegando la tarea en segundo plano.
- **Qué se gana:** El tiempo de respuesta hacia el usuario final disminuye drásticamente, ya que el Gateway responde la confirmación de reserva inmediatamente después del guardado en la base de datos, sin aguardar la latencia SMTP del proveedor de correos.
- **Qué se sacrifica:** Al delegarse de forma volátil dentro del hilo de ejecución y no existir un intermediario, si el servicio de notificaciones se encuentra inactivo en el momento exacto de la llamada, el comprobante se pierde irrecuperablemente.
- **Alternativas descartadas:** Message Broker centralizado (ej. RabbitMQ), descartado en esta fase por introducir sobrecarga infraestructural temprana para un único flujo de eventos.

3.4 API Gateway implementado en FastAPI (Python)

- **Por qué se eligió:** Se optó por desarrollar el Backend For Frontend (BFF) utilizando el framework FastAPI en lugar de lenguajes orientados a objetos tradicionales.
- **Qué se gana:** Soporte nativo para asincronía (`asyncio`), resultando óptimo para un componente cuya única responsabilidad es multiplexar tráfico I/O (recibir solicitudes HTTP REST y despachar llamadas gRPC) sin bloquear los hilos de ejecución.
- **Qué se sacrifica:** FastAPI carece del rendimiento puro de CPU que ofrecen lenguajes compilados, aunque dado que el Gateway es un componente puramente *I/O-bound*, el impacto real en el rendimiento es imperceptible.
- **Alternativas descartadas:** Spring Cloud Gateway (Java), descartado por su excesivo consumo de memoria en inactividad para un microservicio con rol de traductor ligero.

4. Descripción de cada servicio y sus métodos gRPC

El ecosistema expone su funcionalidad interna mediante contratos de Protocol Buffers (`.proto`) estrictamente definidos.

4.1 User Service (Java / Spring Boot)

Administra la persistencia, identidad y seguridad de los perfiles de usuario.

- **CreateUser:** Inserta un usuario aplicando una función criptográfica unidireccional (BCrypt) a su contraseña.
- **AuthenticateUser:** Verifica las credenciales de ingreso frente a los hashes almacenados, retornando el perfil de forma segura.
- **GetUser:** Recupera un perfil de usuario mediante su UUID, método de uso interno para validación de identidades en otros servicios.
- **UpdateUser:** Aplica modificaciones parciales sobre la estructura del perfil almacenado.

4.2 Inventory Service (Go)

Gobierna el catálogo hotelero, tarifas y disponibilidad física concurrente de las habitaciones.

- **SearchAvailableRooms:** Ejecuta búsquedas de lectura intensiva sobre el catálogo aplicando filtros cruzados (fecha, precio, capacidad).
- **UpdateStock:** Aplica la mutación lógica en la disponibilidad, aceptando la acción de "BLOQUEAR" o "LIBERAR" cupo sobre un tipo de habitación específico.

4.3 Reservation Service (Go)

Actúa como el orquestador principal, cruzando identidad de usuarios y control de stock.

- **CreateReservation:** Ensambla la transacción distribuida: verifica al usuario, descuenta el stock en el inventario, persiste el registro final y gatilla el proceso de correo electrónico.

- **GetReservation:** Recupera el estado operativo y los detalles de cobro de una reserva particular.
- **ListReservations:** Extrae el listado histórico de transacciones realizadas en el sistema.

4.4 Notification Service (Python)

Servicio utilitario y de contacto, encargado de conectar el clúster interno con proveedores externos (API Resend).

- **SendConfirmation:** Recibe los datos consolidados en un solo mensaje (`user_id`, `reservation_id`, `tipo`, `email`) y ejecuta el despacho asíncrono, garantizando el registro persistente de la acción de contacto en su base de datos.

5. Descripción de los use cases y cómo fluyen entre servicios

5.1 Registro de Identidad (User Service)

1. **Operación:** El cliente externo solicita la creación de una cuenta suministrando sus datos mediante un payload JSON en la ruta HTTP `POST /users`.
2. **Flujo Técnico:** El API Gateway recibe el JSON, ensambla un mensaje Protobuf y despliega una llamada gRPC al método `CreateUser` del **User Service**. Este último encripta el acceso, persiste la entidad en `users_db` y retorna la respuesta hacia el Gateway, que la transmite al cliente.

5.2 Acceso al Sistema (Login)

1. **Operación:** El usuario envía sus credenciales para autenticarse en la ruta `POST /login`.
2. **Flujo Técnico:** El Gateway canaliza la petición invocando sincrónicamente `AuthenticateUser` vía gRPC en el **User Service**. El servicio extrae la información de su base de datos, ejecuta la comprobación BCrypt en memoria y, de ser válida, expide los metadatos seguros de vuelta por la cadena.

5.3 Auditoría de Inventario (Búsqueda)

1. **Operación:** El cliente despliega filtros de búsqueda para consultar las habitaciones disponibles en un marco de tiempo mediante un `POST /api/inventory/search`.
2. **Flujo Técnico:** El Gateway mapea los argumentos y despliega una invocación gRPC a `SearchAvailableRooms` del **Inventory Service**. El servicio consulta su base de datos `inventario_db`, procesa el cruce de disponibilidad y despacha de retorno un arreglo tipado de opciones disponibles.

5.4 Confirmación de Reserva (Orquestación)

1. **Operación:** El usuario expide la solicitud de cobro y bloqueo sobre una habitación utilizando `POST /reservations`.

2. Flujo Técnico:

- El Gateway redirige la petición gRPC invocando `CreateReservation` en el orquestador **Reservation Service**.
- El orquestador enruta una llamada a `GetUser` (**User Service**) para confirmar la veracidad de la identidad y almacenar su `email`.
- Se efectúa una llamada a `UpdateStock` (**Inventory Service**) para descontar un cupo físico del stock de manera concurrente.
- Se registra el estado definitivo en la base de datos local `reservas_db`.
- Se despacha una *goroutine* de delegación hacia `SendConfirmation` (**Notification Service**) enviando los metadatos y el `email`. El Notification Service registra el intento en su base de datos y conecta con el SMTP externo de manera autónoma y paralela al fin de la transacción.

6. Link Repositorio:

<https://github.com/darkkings19/Proyecto-SD-Reserva-de-Hoteles.git>